Programming Fundamentals in C

Lecture: Bitwise AND, Binary Representation, Bit Masks and ASCII

1. Introduction and Lecture Roadmap

A computer stores integer values in binary. Each binary digit is a bit, and each bit represents a power of 2. In C, bitwise operators allow us to work with individual bits of an integer instead of treating the whole number as one value.

In this lecture, we will use the programs t1.c to t7.c to develop the idea step by step. We will first see bitwise AND, then use individual powers of 2 as masks to inspect bits. We will also connect decimal and binary representations, study ASCII codes of capital and small alphabets, and finally see how a bit can be turned off using a mask.

Topics covered in this lecture

- Binary representation of an unsigned integer using powers of 2
- Converting decimal to binary and binary to decimal using powers of 2
- Bitwise AND (&) and its truth table
- Using powers of 2 as bit masks
- Checking individual bits with if statements
- Checking selected groups of bits
- ASCII codes of capital and small English alphabets
- Why the ASCII difference between corresponding capital and small letters is 32
- Turning a selected bit off using a bit mask

2. Binary Representation and Powers of 2

The binary number system has only two digits: 0 and 1. Each position represents a power of 2. Starting from the rightmost position, the values are 2^0, 2^1, 2^2, 2^3, and so on.

For an 8-bit value, the positions are:

128 64 32 16 8 4 2 1

A bit is 1 when that power of 2 contributes to the number, and it is 0 when it does not.

Example 1: Convert 13 to binary using powers of 2.

The largest power of 2 not greater than 13 is 8. The remaining value is 5. Then use 4, leaving 1, and finally use 1. Therefore:

13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0

So 13 in binary is 00001101.

Example 2: Convert 42 to binary.

42 = 32 + 8 + 2 = 2^5 + 2^3 + 2^1

Therefore 42 = 00101010 in 8-bit representation.

Example 3: Convert 10110101_2 to decimal.

The 1 bits occur at 128, 32, 16, 4, and 1.

$10110101_2 = 128 + 32 + 16 + 4 + 1 = 181_{10}$

The important idea is that binary conversion here is done by identifying powers of 2. We are not using the repeated-division method.

3. Bitwise AND (&)

The bitwise AND operator & compares two integers bit by bit. For each position, the result bit is 1 only when both corresponding bits are 1.

Truth table:

0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1

```
unsigned char x = 13, y = 10;
```

```
printf("%d\n", (x & y));
```

13 = 00001101
10 = 00001010
& = 00001000

Therefore x & y is 8.

This is exactly the idea used in t1.c: two values are ANDed, and the resulting value is printed. The operation does not change x or y; it produces a new integer result.

4. Powers of 2 as Bit Masks

The values 1, 2, 4, 8, 16, 32, 64, and 128 correspond to individual bit positions in an 8-bit value.

Their binary forms are:

1 = 00000001
2 = 00000010
4 = 00000100
8 = 00001000
16 = 00010000
32 = 00100000
64 = 01000000
128 = 10000000

A mask is a value used with a bitwise operation to examine or manipulate particular bits. When we AND a value with one of these masks, all other bit positions become 0, while the selected position remains 1 if it was already on.

Example: Suppose x = 37.

37 = 00100101

x & 4 = 00100101 & 00000100 = 00000100 = 4

Because the result is 4, the third bit is on. If x & 4 produces 0, that bit is off.

5. Checking Individual Bits — t2.c

t2.c generates an unsigned char and checks each of its eight bits separately. The first bit uses mask 1, the second uses 2, the third uses 4, and so on.

```
if ((x & 1)) printf ("First bit is on\n");
```

```
if ((x & 2) == 2) printf ("Second bit is on\n");
```

```
if ((x & 4) == 4) printf ("Third bit is on\n");
```

The first condition is written slightly differently, but the idea is the same: if x & 1 is non-zero, the first bit is on. For the other conditions, the code explicitly compares the result with the mask.

Important: a bit being on means that its value is 1, not that the entire number is odd/even. For example, 12 = 00001100 has the third and fourth bits on, while its first bit is off.

6. Checking Higher Bits — t3.c

t3.c continues the same idea beyond the eighth bit. In an unsigned integer, 256 represents the ninth bit, 512 the tenth bit, 1024 the eleventh bit, and so on.

The sequence continues because powers of 2 continue:

256 = 2^8, 512 = 2^9, 1024 = 2^10, 2048 = 2^11, 4096 = 2^12, 8192 = 2^13.

Thus, the condition (x & 1024) == 1024 asks one precise question: is bit 11 on?

Notice that the program adds 256 to rand(). This guarantees that the resulting value has at least the 256 contribution, so the ninth bit is on. The other higher bits depend on the generated random value.

7. Checking a Group of Bits — t4.c

Sometimes we do not need to inspect each bit separately. t4.c uses the mask 3. In binary, 3 is 00000011, so it keeps only the last two bits.

```
unsigned char result = (x & 3);
```

Possible results are only 0, 1, 2, or 3 because the last two bits can have only four combinations:

00 = 0
01 = 1
10 = 2
11 = 3

Therefore t4.c can classify the last two bits by examining result. This is a useful general pattern: choose a mask whose 1s identify the bit positions you want to keep.

8. ASCII Codes and the 32 Difference

ASCII assigns numeric codes to characters. In standard ASCII, capital English letters A through Z occupy decimal codes 65 through 90, while small English letters a through z occupy 97 through 122.

Examples:

'A' = 65, 'B' = 66, 'C' = 67 ... 'Z' = 90
'a' = 97, 'b' = 98, 'c' = 99 ... 'z' = 122

For every corresponding pair, the small letter is exactly 32 greater:

'a' - 'A' = 97 - 65 = 32
'b' - 'B' = 98 - 66 = 32
'u' - 'U' = 117 - 85 = 32

Why is the difference 32? In ASCII, the codes for the capital and corresponding small letters differ in bit 5, whose value is 32. The remaining relevant bits are arranged so that changing that bit changes the letter case.

For example:

'U' = 85 = 01010101
'u' = 117 = 01110101

The difference is 32. This is why adding 32 to an uppercase ASCII code gives the corresponding lowercase code, and subtracting 32 from a lowercase code gives the corresponding uppercase code, for letters A–Z/a–z.

t5.c prints U and u together and also examines their bits. It gives a concrete example of how ASCII characters are simply integer values that can participate in bitwise operations.

9. Generating a Random Lowercase Letter — t6.c

t6.c uses:

```
unsigned char x = rand() % 26 + 'a';
```

rand() % 26 produces a value from 0 through 25. Adding 'a' shifts this range to the ASCII codes of the 26 lowercase English letters.

So the possible values are:

'a' through 'z', or decimal 97 through 122.

The important connection is that a character such as 'm' is stored as its ASCII integer code. The %c format displays the character, while %d displays the same stored value as a decimal number.

10. Turning a Bit Off — t7.c

t7.c uses the expression:

```
x = (x & (255 - 32));
```

For an 8-bit mask, 255 is 11111111. Since 32 is 00100000, subtracting 32 gives 223, which is:

223 = 11011111

ANDing x with 11011111 forces the bit worth 32 to 0, while the other bit positions are preserved.

For example, if x = 117 ('u'):

117 = 01110101
223 = 11011111
& 01010101 = 85

The result is 85, which is 'U'. This explains an especially interesting connection in t7.c: turning off the bit worth 32 changes a lowercase letter to its corresponding uppercase letter, provided x is a lowercase ASCII letter.

The mask 255 - 32 is used here because 255 has all eight bits on and subtracting 32 makes only the 32-bit position zero.

11. Key Ideas to Remember

- Every binary position represents a power of 2.
- Decimal to binary can be done by selecting powers of 2 that add up to the decimal value.
- Binary to decimal is done by adding the powers of 2 corresponding to the 1 bits.
- Bitwise AND (&) produces 1 at a position only when both input bits are 1.
- A power of 2 can be used as a mask to inspect one particular bit.
- Mask 3 (00000011) keeps the last two bits.
- ASCII capital A–Z are 65–90 and lowercase a–z are 97–122.
- The corresponding lowercase ASCII code is 32 greater than the uppercase code.
- Bit value 32 is 2^5, so it is the sixth bit when counting the rightmost bit as the first bit.
- ANDing with a mask containing 0 at a selected position and 1 elsewhere can turn that bit off.